

李乐岩

3073751449@qq.com | +86 18805506898 | GitHub: <https://github.com/lionlepower>

爱丁堡大学 **University of Edinburgh** 英国 | 2025.09 – 2026.09 预计

高性能计算与数据科学 硕士

利物浦大学 **University of Liverpool** 英国 | 2023.09 – 2025.06

计算机科学 本科, 西交利物浦大学 2+2 项目, 一等荣誉学位

项目经历:

面向 HPC Benchmark 的 AI Agent 性能分析系统

项目描述: 该项目构建一个面向高性能计算实验场景的 AI Agent 系统, 用于辅助管理 PETSc benchmark 实验、解析 Slurm / PETSc 运行日志、查询历史性能数据、检索 HPC 知识库, 并自动生成 Markdown 格式性能分析报告。系统将 Agent tool calling、RAG 检索、数据库存储、Redis 缓存、后端 API 和可观测性机制结合起来, 使用户能够通过自然语言完成实验结果查询、并行配置对比、性能瓶颈分析和报告生成。

技术栈:

Python、FastAPI、LangChain / LlamaIndex、FAISS / Chroma、OpenAI API / 本地 LLM、SQLite / MySQL、Redis、Pandas、Matplotlib、PETSc、MPI、OpenMP、Slurm、Docker、Linux

工作内容:

- 设计 AI Agent 系统整体架构, 将系统拆分为 API 接入层、Agent 规划层、工具调用层、RAG 检索层、数据库层、缓存层、报告生成层和可观测性模块, 使复杂性能分析任务能够被拆解为多个可执行步骤。
- 使用 FastAPI 构建后端服务, 提供自然语言问答、实验结果查询、日志上传解析、历史记录检索、性能指标计算和 Markdown 报告生成等接口, 并统一处理参数校验、异常返回和响应格式。
- 设计 Agent tool calling 流程, 使系统能够根据用户问题自动选择数据库查询、日志解析、指标计算、知识库检索、图表生成和报告生成等工具, 完成多步骤任务编排。
- 构建 RAG 检索流程, 将 PETSc、MPI、OpenMP、Slurm、Linaro MAP 和个人实验笔记纳入知识库, 完成 Markdown 文档切分、embedding 生成、向量索引构建和 top-k 语义检索。
- 设计实验结果数据库表结构, 存储 benchmark 配置、运行状态、性能指标、错误日志和分析记录, 并支持按 problem size、execution mode、total cores、created_at 等字段查询历史实验结果。
- 针对常见查询场景设计索引策略, 例如按问题规模和核心数查询最优配置、按执行模式对比 MPI-only 与 hybrid 结果、按时间范围追踪实验记录, 为后续查询优化和慢查询分析预留空间。
- 引入 Redis 缓存热点自然语言查询、常用实验统计结果、短期会话状态和 query hash 对应的分析结果, 通过 TTL 控制缓存生命周期, 减少重复数据库查询、向量检索和 LLM 调用开销。
- 编写 Slurm / PETSc 日志解析模块, 从原始输出中提取 problem size、nodes、MPI ranks、OpenMP threads、total cores、runtime、execution mode、solver status 和 error message 等字段, 并转换为结构化实验数据。
- 使用 Pandas 和 Matplotlib 实现性能指标计算与可视化模块, 支持自动计算 runtime、speedup、parallel efficiency, 并生成不同问题规模和并行配置下的对比分析图表。
- 为每次 Agent 调用生成 TraceID, 记录用户问题、工具调用链路、检索文档、缓存命中情况、数据库查询耗时、RAG 检索结果和 LLM 响应耗时, 用于排查检索失败、工具调用错误和接口性能问题。
- 设计报告生成流程, 将实验数据、图表、RAG 检索结果和 Agent 分析结论整合为 Markdown 格式性能分析报告, 支持沉淀到 Obsidian 或 GitHub 文档中。
- 基于真实 PETSc benchmark 数据进行验证, 支持分析 2D / 3D stencil 问题在 MPI-only 与 hybrid MPI+OpenMP 配置下的 runtime、speedup、parallel efficiency 和性能瓶颈差异。

可关联面试知识点:

AI Agent、tool calling、RAG、向量检索、HTTP、REST API、数据库索引、事务、Redis 缓存、缓存一致性、进程与线程、异步任务、文件 I/O、日志追踪、系统设计、HPC 性能分析。

多智能体 Actor-Critic 方法中的价值函数分解研究

该项目研究将 VDN、QMIX 等价值函数分解方法引入 MAPPO 框架, 探索 factorised critic 在 cooperative multi-agent environments 中对协作效率、收敛稳定性和策略表现的影响。该项目可作为 AI Agent 方向中多智能体协作、任务分解和策略学习的算法基础展示。

技术栈：

Python、PyTorch、ePyMARL、PyMARL、MAPPO、VDN、QMIX、多智能体强化学习

工作内容：

- 基于 ePyMARL / PyMARL 框架实现 MAPPO、MAPPO-VDN、MAPPO-QMIX 等多智能体强化学习算法变体，并完成训练配置、实验运行和结果评估。
- 修改 actor-critic 框架中的 critic 网络结构，引入 VDN 风格的加性价值分解和 QMIX 风格的单调混合网络，探索 factorised critic 在 cooperative multi-agent tasks 中的适用性。
- 支持 centralized training with decentralized execution 训练范式，在训练阶段利用全局状态和联合动作信息，在执行阶段由各 agent 基于局部观测独立决策。
- 在 Matrix Game 和 Predator-Prey 环境中设计并完成对比实验，评估不同算法在 test return、收敛速度、最终回报和协作表现上的差异。
- 构建实验日志记录与结果分析流程，对 reward curve、test return、convergence behaviour 和 training stability 等指标进行可视化和比较。
- 分析价值函数分解方法应用于 actor-critic 框架时可能出现的不稳定问题，包括 critic approximation error、多智能体非平稳性、credit assignment 和 mixing network 表达能力限制。
- 完成完整学术论文撰写，系统整理算法设计、代码实现、实验结果、局限性与后续改进方向。

可关联面试知识点：

强化学习、多智能体协作、PyTorch、模型训练、实验评估、任务分解、非平稳性、credit assignment。

技术能力

编程语言与工程开发： 熟悉 Python、C/C++，掌握 SQL 基础，了解 JavaScript、HTML、CSS；能够使用 Python 完成 AI 应用原型开发、后端接口实现、数据处理、日志解析、实验自动化和模型训练流程。

AI Agent 与 LLM 应用： 了解 LLM Agent 基本架构，包括 tool calling、任务分解、RAG 检索增强生成、prompt engineering、记忆管理、多轮对话状态管理和 Markdown 报告生成；具备将大模型能力与外部工具、文件系统、数据库、缓存和自动化脚本结合的项目开发经验。

RAG 与向量检索： 了解 Markdown 文档解析、文本切分、embedding 生成、FAISS / Chroma 向量索引、top-k 语义检索和上下文拼接流程；能够围绕领域知识库构建检索增强问答与报告生成系统。

后端与系统设计基础： 了解 FastAPI、REST API、HTTP、客户端与服务端交互、参数校验、异常处理、日志追踪、TraceID、任务调度、限流、幂等性和可观测性等后端工程概念。

数据库与缓存： 了解 MySQL / SQLite 基础使用，理解索引、事务、锁、查询优化等数据库基础概念；了解 Redis 常用缓存场景，包括热点查询缓存、短期会话状态、TTL、缓存穿透、缓存击穿和缓存一致性。

机器学习与数据分析： 熟悉 PyTorch，具备深度学习和多智能体强化学习算法实现与实验评估经验；熟悉 Pandas、NumPy、Matplotlib，可完成结构化数据处理、指标计算、实验日志分析和可视化。

工具与开发环境： 熟悉 Linux、Bash、Git、VS Code、Docker 基础操作；具备使用自动化脚本管理实验、处理日志、维护项目文档和构建可复现开发流程的经验。

论文发表

Li, Leyan. Convolutional Neural Networks Based Medical Image Analysis.
International Conference on Engineering Management, Information Technology and Intelligence, EMITI 2024.